

Session 6: Knowledge Representation & Reasoning

Last session:

- The “magic” of constraint propagation

This session:

- Augmenting a constraint network with knowledge representations (a “**knowledge base**”), and augmenting basic propagation mechanisms with reasoning strategies (an “**inference engine**”).

Key idea: intelligence requires first getting to know what the problem is. Hence, assigning credit = an epistemological process

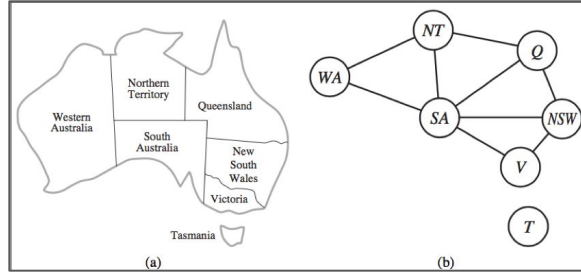
		6	9			4		
	4				7	3		2
3	1						8	
1		3		2	3	6		5
				2				
	9						3	
9	7							4
		4	8			2	9	
	6				4	8		

Heuristic approach: problem is given: we know how to propagate, and if we get stuck, we can always just guess

4	.	.	.	.	.	8	.	5
.	3	.	.	.	.	.	.	.
.	.	.	7	.	.	.	.	.
.	2	.	.	.	.	.	6	.
.	.	.	.	8	.	4	.	.
.	.	.	.	1	.	.	.	.
.	.	.	6	.	3	.	7	.
5	.	.	2	.	.	.	.	.
1	.	4	.	.	.	.	.	.

Epistemological approach: imagine we don't even know this is Sudoku.
How do we find out?

John McCarthy: we get to know problems based on our knowledge of stable phenomena and ability to reason about specific situations



4	.	.	.	.	.	8	.	5
.	3	.	.	.	.	.	.	.
.	.	.	7	.	.	.	.	.
.	2	.	.	.	.	.	6	.
.	.	.	.	.	8	.	4	.
.	.	.	.	.	1	.	.	.
.	.	.	6	.	3	.	7	.
5	.	.	2	.	.	.	.	.
1	.	4	.	.	.	.	.	.

- **Phenomena:** e.g., knowledge of games, propagation, in general, etc.



- **Situations:** e.g., reasoning about what the constraints are on a particular Sudoku game.

McCarthy's core idea

- Argued intelligence depends on representing **commonsense knowledge about the world** and reasoning about this knowledge.
- This knowledge should not be tied to a single problem.
- Knowledge should be reusable across many different situations.



John McCarthy (1927-2011),
co-founder of MIT AI lab and
founder of Stanford AI lab

Search trees, constraint networks etc. are not stable, since situations change. Phenomena are stable.

Situations (states of the “universe”)

- A specific context in which problems arise and actions occur.
- Situations include current facts, goals, and environment.

Phenomena (stable facts and concepts)

- General, stable facts about how the world works.
- Examples include physical and everyday regularities.

*Learning to assign credit = learning to **represent** and **reason** about **knowledge** of **phenomena** regarding changing, real-world **situations**.*

Supermarket example: how to send targeted ads to a shopper who visits rarely, but always buys frozen spinach?

- Standard way: use data science to analyze data in receipts + customer info.
- McCarthy's way: use commonsense knowledge to interpret behavior:
 - The shopper likely has a freezer.
 - The shopper likes the frozen spinach more than the place she/he usually shops at.

Supermarket example: McCarthy argues KRR = greater robustness, efficiency

- **Robustness** to changes in customer's "situation":
 - How much space is currently in the shopper's freezer, whether relatives are visiting, etc.
- **Efficiency:**
 - Standard data science: only works well with vast volumes of data + big computers to analyze data (brute force)
 - McCarthy's way (KRR): just need a representation of commonsense knowledge + a few facts about a customer.

How could an AI system or org. reason about the vast number of possible situations?

- McCarthy's answer: this is possible if we can represent **commonsense knowledge** about phenomena and situations, and some way of **reasoning** about this knowledge.
- Before we talk about what these mean, let us talk about the nature of credit assignment in KRR.

Credit assignment & knowledge representation and reasoning

- (1) The vision: an AI system is intelligent when it behaves like an “**Advice taker**”
- (2) The credit assignment challenge of the “Advice taker” = the “**frame problem**”

McCarthy's "Advice taker" vision (1959)

Advice taker = a hypothetical AI system. Think of an LLM... but (in McCarthy's view at least), something much better. Some criteria:

- (1) It learns new representations of problem just by you telling it **facts** (e.g., not prompts!) about the situation in natural language (English, Korean, etc.) = it takes advice
- (2) It represents knowledge as **concepts** and **facts**, not procedures, algorithms.
- (3) It precisely and logically reasons about these concepts and facts to modify its representations (e.g., no hallucination, no slop, no generic outputs).

McCarthy's "Advice taker" - Requirements

- Ability to represent commonsense knowledge about any relevant phenomena
- Ability to reason about this knowledge across situations

The frame problem

What are the challenges of achieving McCarthy's Advice taker vision?

- Obvious one: there is a lot of commonsense knowledge about the world

McCarthy: actually the challenge is much more severe

- We need knowledge not just of what to do and when to do it
- We need knowledge of what not to do = the frame problem

Illustrating the frame problem - what did Ahn Junghwan need to “know” to score a goal against Italy? (Daejeon, World Cup 2002)



Solving the frame problem

Requires:

- Ability to represent commonsense knowledge about any relevant phenomena
- Ability to reason about this knowledge across situations

Representing commonsense knowledge: how can we do this efficiently given how much of it there is?

- To see how, let us distinguish explicit vs. implicit knowledge

Consider the following sentence: why do you believe it?

“The Pyramids of Giza in Egypt are older than my neighbor’s house”

Consider the following sentence: why do you believe it?

“The Pyramids of Giza in Egypt are older than my neighbor’s house”

→ Answer: it was **logically entailed**, meaning that it was implicit in your commonsense knowledge even though you never explicitly thought about it before.

Implicit knowledge: implications for representing commonsense knowledge

- (1) Knowledge representations should be **declarative**
- (2) Knowledge representations need to allow **concepts**
- (3) Knowledge representations need to reflect **contexts**

(1) Knowledge representations need to be declarative

- What was implicit was based not on sequences (e.g., sequences of words in that sentence), but **facts** and **concepts**.
- Hence, commonsense knowledge should be represented **declaratively** (e.g., as **lists**) not procedurally (e.g., as algorithms — most programming languages).

Declarative representations - omelette example: can more easily change the “recipe” on the right than the one on the left



Procedural representation: step-by-step sequence — changing one step may change the whole recipe.



Declarative representation: changing one “fact” may still enable changing the overall recipe fairly simply.

(2) Knowledge representations need to allow concepts

- **Concepts.** Not just facts or programs: ways of talking about problems at different **levels of abstraction**.
- Our implicit knowledge told us that the Pyramids of Giza should be older than my neighbor's house since we have abstract concepts of pyramids and of a neighbor's house.

(3) Knowledge representations need to reflect contexts

- **Contexts.** the meaning of our knowledge representations depends on our situations and goals — the context.
- Consider the following sentence:

“I tried to fill the suitcase with the whale, but it was too big”.

- What is “it”? How do you know?

(3) Knowledge representations need to reflect contexts

- **Speech acts:** knowledge representations can be used to indicate actions to be performed. In this case, the meaning of the knowledge representation depends on who is using it (their goals, etc.).
- Example:
 - Consider the phrase, “Apple (the company) needs to perform better”.

Elaboration: adding or removing facts, concepts etc. from a knowledge base

- Elaboration involves “**operations**” on a knowledge base
 - Syntactic operations: putting knowledge in the correct format.
 - Semantic operations: modifying which concepts or facts are valid or relevant.
 - These operations are costly.
 - “Garbage” knowledge representations can easily pile up in a knowledge base.
 - McCarthy: intelligence requires some way to efficiently “take out the trash” = “**elaboration tolerance**”

Elaboration tolerance

The effort required to add new [facts and concepts] to the [knowledge base] is proportional to the complexity of the [facts and concepts].

Low elaboration tolerance is akin to “digital clutter”

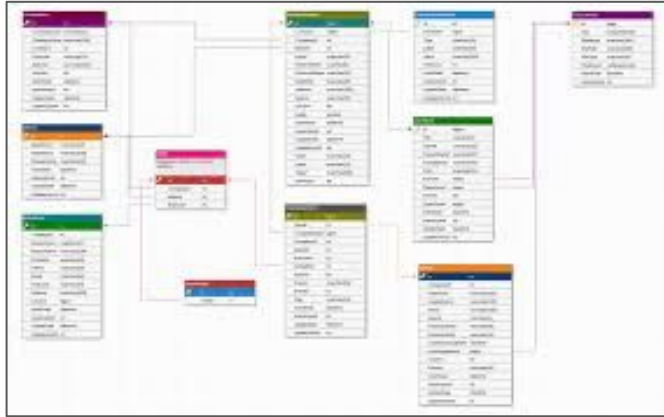


Elaboration tolerance requires **default reasoning** = making commonsense assumptions, even if we are not sure

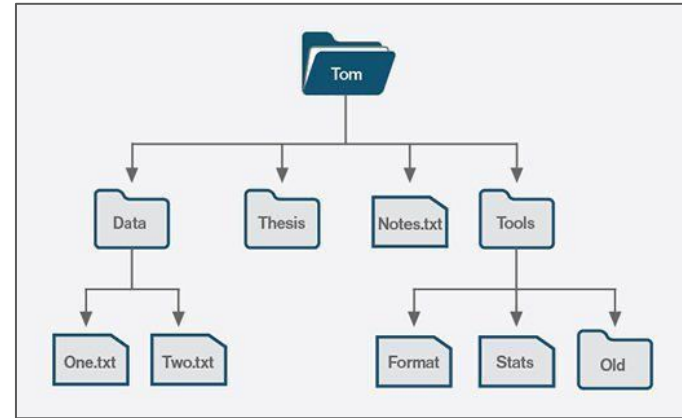
Example: “*I tried to fill the suitcase with the whale, but it was too big*”

→ It *could* be a stuffed animal, or a baby whale, or a lunchbox with a picture of whale on it... but we need to make some assumption to simplify things.

Elaboration tolerance requires adequately **expressive languages**.
Example: SQL database versus file system.



SQL = express logical
relations to enable high
data integrity



File system = lower
expressiveness, and
lower data integrity.

Approaches to knowledge representation and reasoning

(1) Deep learning approach

(2) Mathematical logic approach

(1) Deep learning approach to KRR

- Knowledge representation:
 - Bit string patterns extracted from large volumes of data and stored in matrix-like structures (vector databases, knowledge graphs, etc.)
 - Concepts = statistical relations among the bit strings
 - Context = adding files using techniques such as retrieval-augmented generation (RAG) and context engineering.
- Reasoning: heuristics for prompting
 - E.g.: chain-of-thought procedures

(1) Deep learning approach to KRR (continued)

- Reasoning: heuristics for prompting
 - E.g.: chain-of-thought procedures
- Limits:
 - So far: little support for solving the frame problem or elaboration tolerance

(2) Mathematical logic approach (McCarthy's approach)

- Mathematical logic = precise, formal languages for describing facts and concepts etc. and how to reason about them.
 - Often caricatured as just “if-then rules”.
- Knowledge is represented using rich conceptual structures (e.g., **ontologies**).
 - The goal is to make knowledge transparent to reason step-by-step, not heuristically through prompting.

(2) Mathematical logic approach (Palantir example)

The screenshot displays the Palantir Foundry Ontology Manager interface. At the top, there are tabs for 'Archetypes' and 'Ontology', with 'Ontology' being the active tab. Below the tabs, a breadcrumb trail shows 'Back' and 'Properties of [Example Data] Aircraft'. The main area is divided into three panels:

- Left Panel:** A list of properties for the 'aircraft' archetype, including 'tail_number', 'serial_number', 'manufacture_year', 'manufacturer', 'model', 'number_of_seats', 'capacity_in_pounds', 'unique_carrier_name', 'carrier_code', 'operating_status', and 'aircraft_status'.
- Middle Panel:** A list of 14 properties for the '[Example Data] Aircraft' archetype. The 'Acquisition Date' property is highlighted in blue. Other properties include 'Aircraft Status', 'Capacity In Pounds', 'Carrier Code', 'Display Name', 'Manufacture Year', and 'Manufacturer'.
- Right Panel:** A detailed view of the 'Acquisition Date' property. It shows the 'PROPERTY ID' as 'acquisition_date', the 'DISPLAY NAME' as 'Acquisition Date', and the 'DESCRIPTION' as 'Description'. The 'RID' is 'ri.ontology.main.property.9ef65fb0-1...', and the 'STATUS' is 'Active'.

At the bottom of the interface, there is a button labeled 'Activate → Ontology Manager' and a search icon.